



Universidad
de Valparaíso
CHILE

Manual de Usuario

Transformador XML-JATS

Universidad de Valparaíso

Facultad de Medicina

Autor: Cristian Carreño León

Fecha: 15-03-2026

Versión: 0.68

Contacto: cristian.carreno@uv.cl

Documentación General (README)

Transformador XML JATS (JATS XML Transformer) - v0.68

Una herramienta avanzada impulsada por Inteligencia Artificial para convertir documentos de Word (.docx) a formato JATS XML validado, diseñada específicamente para el flujo editorial de revistas científicas.

Esta aplicación automatiza el proceso de etiquetado semántico, extracción de tablas e imágenes, y validación contra el estándar NISO JATS Version 1.4 (ANSI/NISO Z39.96-2024).

Características Principales

- **Conversión Inteligente:** Utiliza LLMs (Google Gemini 2.5 Flash) para interpretar la estructura lógica del documento y generar etiquetas JATS precisas.
- **Soporte Multiformato:** Procesa documentos Word (.docx) y PDF (.pdf).
- **Extracción de Metadatos:** Identifica y extrae automáticamente metadatos clave (título, autores, DOI, fechas de publicación, recepción y aceptación).
- **Extracción de Tablas:** Las tablas del documento se convierten automáticamente a <table-wrap> con <thead>/<tbody> correctamente estructurados.
- **Preservación de Texto:** El sistema etiqueta el texto original sin modificarlo respetando el trabajo de los correctores humanos.
- **Revisión Interactiva:** Permite editar metadatos y dialogar con un chatbot para completar información faltante antes de la generación.
- **Validación JATS 1.4:** Valida contra el último estándar NISO JATS Version 1.4 (ANSI/NISO Z39.96-2024).
- **Conversión a HTML:** Genera archivos HTML autocontenidos con logo incrustado (Base64), tema claro/oscuro, tabla de contenidos interactiva y enlaces funcionales. Las referencias bibliográficas se renderizan correctamente con DOIs clickeables.
- **Vista Previa en Nueva Pestaña:** La previsualización del HTML se abre en una pestaña del navegador con soporte completo de UTF-8 y navegación por anclas.
- **Una Sola API Key:** La clave de API se guarda de forma segura en una base de datos SQLite local. No es necesario reingresarla en cada sesión. Google gestiona automáticamente el paso entre tier gratuito y tier de pago con la misma clave cuando la cuenta tiene facturación habilitada.
- **Monitoreo de Tokens:** Panel integrado que muestra el consumo de tokens acumulado, requests diarias vs. límites del tier gratuito, y alertas automáticas al acercarse al límite.
- **Detección de Cuota Agotada:** Cuando se agota la cuota gratuita, el sistema lo detecta automáticamente y muestra un banner Cuota gratuita agotada usando tier de pago en la barra lateral.
- **Interfaz Dual:**

- Web UI (Streamlit): Interfaz gráfica amigable para arrastrar y soltar archivos, editar contenido extraído y previsualizar resultados.
- CLI (Línea de Comandos): Para automatización y procesamiento por lotes.

Requisitos del Sistema

- Windows, macOS o Linux.
- [Python 3.9+](<https://www.python.org/downloads/>)
- Una clave de API de Google Gemini (Google AI Studio).

Instalación

1. Clonar el repositorio (si aplica) o descargar el código fuente.
2. Crear un entorno virtual (recomendado):

```
python -m venv .venv
```

Activar:

Windows: `.venv\Scripts\activate`

macOS/Linux: `source .venv/bin/activate`

3. Instalar dependencias:

```
pip install -r requirements.txt
```

4. Configurar la API Key de Gemini (elige una opción):

Opción A Desde la interfaz web (recomendado):

Al abrir la aplicación, dirígete a la pestaña " Configuración" en el menú izquierdo, pega tu clave en el campo correspondiente y presiona " Guardar". La clave se almacena de forma persistente y se carga automáticamente en futuras sesiones.

Opción B Variable de entorno (fallback):

```
export GEMINI_API_KEY="tu_clave_aqui"
```

> Nota sobre cuota pagada: si tu cuenta de Google AI tiene facturación habilitada, la misma clave funciona en el tier de pago una vez agotada la cuota gratuita. No se necesita ninguna clave adicional.

Uso

Interfaz Web (Recomendado)

La interfaz gráfica es la forma más fácil de usar la herramienta.

1. Ejecuta la aplicación:

```
streamlit run streamlit_app.py
```

2. Abre tu navegador en la URL mostrada (usualmente `http://localhost:8501`).
3. Sube tu archivo `.docx` o `.pdf`.
4. Revisa y completa los metadatos extraídos.
5. Genera el XML, valida y descarga los resultados.
6. Genera el HTML y haz clic en "Ver vista previa en nueva pestaña" para inspeccionar el resultado.

Línea de Comandos (CLI)

Para usuarios avanzados que deseen integrar la herramienta en scripts.

Transformación (Word → XML):

```
python -m modules.transformer entrada.docx salida.xml
```

Conversión (XML → HTML):

```
python -m modules.xml_html entrada.xml salida.html
```

Estructura del Proyecto

- `streamlit_app.py`: Punto de entrada de la aplicación web (Streamlit UI).
- `modules/`:
- `transformer.py`: Núcleo de la lógica de conversión. Desacoplado de la interfaz de usuario. Usa `tenacity` para manejo robusto de reintentos (HTTP 429) y control de errores. Soporta generación de hasta 65K tokens de salida. Retorna `quota_exceeded: True` cuando se agota el límite diario gratuito.

- `metadata_processor.py`: Módulo para la extracción de metadatos mediante IA, agnóstico al entorno gráfico.
- `correction.py`: Módulo para la corrección asistida por IA (preserva texto original), completamente separado del estado de Streamlit.
- `prompts.py`: Repositorio centralizado de instrucciones maestras (prompts) de IA con plantillas detalladas para tablas, secciones, DOI y fechas.
- `xml_html.py`: Convertidor de JATS XML a HTML5 responsivo con renderización completa de referencias bibliográficas (DOIs clickeables, nombres de autores, fuentes en itálica).
- `config_store.py`: Almacenamiento persistente de configuración (API Key, uso de tokens) mediante SQLite. Gestiona una única API key; las funciones *_pro están obsoletas.
- `views/`: Vistas de la interfaz gráfica (transformador, manual de usuario, documentación).
- `data/`: Base de datos SQLite local (`config.db`) excluida de Git.
- `JATS-Publishing-1-3-MathML3-DTD/`: Archivos DTD locales para validación offline.
- `imagenes_extraidas/`: Directorio temporal donde se guardan las imágenes extraídas del documento Word.
- `resources/`: Logo de la revista y recursos gráficos.

Créditos

Desarrollado por Cristian Carreño León\
Escuela de Obstetricia y Puericultura\
Facultad de Medicina\
Universidad de Valparaíso, Chile.

Desarrollado para la Universidad de Valparaíso con el objetivo de optimizar los procesos de publicación científica.

Tecnologías Utilizadas

Tecnología	Uso
Python 3.9+	Lenguaje base
Streamlit	Framework de interfaz web
Google Gemini	Modelo de lenguaje (LLM) para etiquetado inteligente
lxml	Procesamiento, validación y parsing de XML/HTML
python-docx	Extracción de contenido desde archivos Word
FPDF2	Generación de manuales en PDF
JATS 1.4 (ANSI/NISO Z39.96-2024)	Estándar de etiquetado XML

Licencia

Este software es de uso exclusivo para la Universidad de Valparaíso. Todos los derechos reservados.

El código fuente y la documentación contenidos en este repositorio son propiedad intelectual de la Universidad de Valparaíso y su autor. Queda prohibida su reproducción, distribución o modificación sin autorización expresa.

Para consultas sobre licenciamiento o uso, contactar a:
cristian.carreno@uv.cl

Historial de Versiones (Changelog)

v0.68 Metadatos de Revista Persistentes, Selectores DTD y Corrección SSL

- Metadatos de Revista Persistentes: Nueva sección en configuración para guardar datos estructurales de la publicación (título de la revista, editorial, ISSN), los cuales se inyectan automáticamente en cada nuevo flujo ahorrando escritura manual repetitiva.
- Selector de Versión JATS Dinámico: El usuario puede escoger la versión objetivo del estándar (1.3 o 1.4) en la misma página de configuración. El transformador inyectará condicionalmente la declaración `<!DOCTYPE>` y validará contra el DTD JATS oficial preciso usando rutas dependientes del contexto.
- Validación SSL Opt-out Local: Se resolvió un bug prevalente en macOS (SSL: CERTIFICATE_VERIFY_FAILED) ignorando programáticamente fallos SSL intermitentes al hacer el fetch oficial del DTD JATS a NLM. Esto asegura que la herramienta arranque exitosamente en cualquier estado de certificados Python.
- UI Responsiva y Textos Justificados: Mejorada radicalmente la integración HTML CSS para IFrames como OJS (width: 100%) y se aseguró uniformidad con justificación de texto automática en artículos.

v0.67 Unificación a Clave API Única y Detección de Cuota

- Una sola API Key: Eliminado el concepto de "Key Pro" separada. El sistema ahora usa una única clave de API. Google gestiona internamente el paso de tier gratuito a tier de pago cuando la cuenta tiene facturación habilitada en Google Cloud.
- Detección de Cuota Agotada: Cuando transformer.py o correction.py reciben un error 429 tras reintentos, retornan `{'quota_exceeded': True}` en lugar de intentar cambiar de clave. La UI muestra automáticamente un banner Cuota gratuita agotada usando tier de pago en la barra lateral.
- Tabla de Cuotas Simplificada: La sección " Configuración" reemplaza las columnas "Pro" de la tabla por una única columna "Tras cuota libre: Pay-as-you-go". Se añade un panel informativo explicando el modelo

de una sola clave con enlace a [Google AI pricing](https://ai.google.dev/pricing).

- Funciones Obsoletas Marcadas: `save_api_key_pro`, `load_api_key_pro` y `delete_api_key_pro` en `config_store.py` emiten `DeprecationWarning` si se llaman. Se mantienen solo para compatibilidad con bases de datos antiguas.
- Backend Limpio: `transformer.py` y `correction.py` ya no aceptan el parámetro `api_key_pro` en sus firmas. Args residuales se absorben con `_kwargs`.

v0.66 Interfaz Minimalista, Validación de API y Configuración Dedicada

- Página de Configuración Exclusiva: Nueva pestaña " Configuración" dedicada exclusivamente a visualizar y gestionar las API Keys y el modelo IA seleccionado, liberando espacio en el menú principal.
- Validación Robusta de API Keys: Se implementó una verificación exhaustiva de la API Key en el backend, la cual ahora detecta placeholders o claves inválidas antes de enviar peticiones a la API, evitando errores confusos y cierres inesperados.
- Interfaz del Menú Lateral Minimalista: Rediseño del panel lateral en el transformador principal. El estado de la API Key ahora se muestra de forma compacta (mini indicador) bajo el seleccionador de modelo. Las instrucciones pasaron de ocupar gran espacio a mostrarse en un práctico "popover" (ventana flotante).
- Recuperación Automática de XML Crítico (Resilient Parsing): El parser XML ahora cuenta con capacidad de recuperación ante etiquetas malformadas menores (`recover=True`), lo cual permite generar un HTML de vista previa incluso si la IA cometió ligeros errores estructurales en el etiquetado JATS.
- Mejores Mensajes de Error: Si el XML está demasiado dañado para generar HTML seguro, la aplicación ahora detecta el fallo y sugiere amablemente al usuario utilizar la pestaña de "Validación y Corrección" asistida por IA para solucionarlo automáticamente.

v0.65 Auditoría Exhaustiva y Corrección de Bugs Críticos

- 15+ Bugs Críticos Corregidos: Auditoría completa del código fuente de todos los módulos con corrección de errores funcionales que impedían el flujo correcto.
- Extracción de Tablas Mejorada: Instrucciones detalladas en el prompt para generar `<table-wrap>` con `<table>`, `<thead>`, `<tbody>`, `<tr>`, `<th>`, `<td>` completos. Las tablas ya no se omiten ni se generan vacías.
- DOI y Fechas Obligatorios: El DOI y la fecha de publicación son ahora campos obligatorios en la validación de metadatos. Instrucciones reforzadas para que el modelo siempre incluya `<pub-date>` y `<article-id pub-id-type="doi">`.
- Secciones con Títulos: Instrucciones mejoradas para que cada sección del artículo (Introducción, Métodos, Resultados, etc.) se mapee correctamente a `<sec><title>`.
- Token Limit: `max_output_tokens` aumentado de 8192 a 65536 los artículos extensos ya no se truncan.
- Corrección de Modelo: Todos los módulos ahora usan `gemini-2.5-flash` de forma consistente.
- Parsing de Metadatos Robusto: Nuevo método `_normalize_metadata()` que corrige variaciones en las claves del JSON devuelto por la IA. Extracción de JSON mejorada para manejar respuestas con prefijos

no-JSON.

- Corrección de Chatbot: El flujo de corrección por chatbot ahora detecta correctamente XML directo (sin backticks) en las respuestas de la IA, resolviendo el bug donde las correcciones nunca se aplicaban.
- Renderización de Referencias HTML: `xml_html.py` ahora maneja correctamente elementos JATS de bibliografía (`<name>`, `<surname>`, `<given-names>`, `<person-group>`, `<source>`, `<pub-id>`, etc.) con DOIs clickeables y fuentes en itálica.
- Eliminación de Código Duplicado: Removida la definición duplicada de `validar_jats_xml` y la llamada a `construir_prompt_avanzado` (función inexistente).

v0.64 Refactorización del Backend y Desacoplamiento de UI

- Arquitectura Desacoplada: Eliminada la dependencia de `streamlit` (`st.session_state`) en los módulos del backend (`metadata_processor.py`, `correction.py`, `transformer.py`). Ahora toda la configuración (API keys, versión de modelo) se pasa explícitamente, permitiendo ejecución pura por CLI o testing automatizado.
- Centralización de Prompts: Creación del nuevo módulo `modules/prompts.py` que almacena todas las plantillas de instrucciones complejas para la IA, limpiando el código funcional y facilitando la afinación del comportamiento del modelo.
- Reintentos Robustos con Tenacity: El bucle manual de reintentos en las llamadas a la API de Google Gemini ha sido reemplazado por la librería estándar `tenacity`. Ahora el sistema maneja automáticamente los errores de cuota (HTTP 429) usando retardos exponenciales, mientras preserva el fallo rápido para errores irre recuperables como bloqueos por `*Recitation*/Copyright`.

v0.63 API Key Persistente y Monitoreo de Tokens

- Almacenamiento Persistente de API Key: La clave de Gemini se guarda de forma segura en una base de datos SQLite local (`data/config.db`).
- Seguridad de la Key: Una vez guardada, la clave desaparece de la interfaz. Solo se muestra una versión enmascarada (`AlzaxY4Z`). Botones para cambiar o borrar la key en cualquier momento.
- Panel de Uso de Tokens: Nuevo panel expandible en la barra lateral que muestra:
 - Requests usadas hoy vs. límite diario (RPD) con barra de progreso.
 - Tokens consumidos hoy.
 - Límites por minuto (TPM y RPM) del modelo seleccionado.
 - Historial acumulado total.
- Advertencias de Límite: Alertas automáticas al alcanzar el 80% y 100% del límite diario del tier gratuito.
- Registro Automático: Cada operación con IA (generación, corrección, chatbot) registra automáticamente los tokens consumidos.
- Nuevo Módulo `config_store.py`: Gestión centralizada de configuración y métricas mediante SQLite.

v0.62 Mejoras de HTML, Manejo de Recitación y Deduplicación de Referencias

- Deduplicación de Referencias: Corregido el problema donde los números de referencias bibliográficas aparecían triplicados en el HTML (por , <label> y texto de la cita). Nueva función `_strip_leading_ref_number()` limpia automáticamente los números duplicados.
- Vista Previa en Nueva Pestaña: La previsualización HTML ahora se abre en una pestaña nueva del navegador en lugar de un iframe incrustado, resolviendo problemas con enlaces y estilos.
- Corrección UTF-8: Implementación correcta de decodificación UTF-8 en la vista previa (TextDecoder en lugar de atob), solucionando caracteres acentuados corruptos (ej. "crÃáticas" "críticas").
- Navegación por Anclas: Script de interceptación de clicks en enlaces `#ancla` usando `scrollIntoView()`, corrigiendo la navegación interna en páginas abiertas con `document.write()`.
- Preservación de Texto (Prompt): Prompt de generación JATS completamente reescrito para prohibir modificaciones al texto original la IA solo etiqueta, nunca reescribe.
- Manejo de Recitación: Manejo robusto de errores `finish_reason=4` (recitación) de la API de Gemini con reintentos automáticos y temperatura progresiva.
- Filtros de Seguridad Desactivados: Todos los filtros de seguridad de Gemini configurados en `BLOCK_NONE` (apropiado para contenido académico publicado).
- Instrucciones de Referencia en Prompt: Las instrucciones JATS ahora especifican explícitamente que el número de referencia va SOLO en <label>, no en <element-citation>.

v0.6 Actualización JATS 1.4 + Metadatos

- Estándar JATS 1.4: Actualización completa del motor de validación y prompts al estándar ANSI/NISO Z39.96-2024.
- Soporte PDF: Ahora es posible cargar archivos .pdf para extracción de texto y metadatos.
- Módulo de Metadatos: Nueva lógica de extracción y validación de metadatos (título, autores, DOI, fecha).
- UI de Revisión: Nueva sección en la interfaz para editar metadatos y chatbot asistente para completar información faltante.

v0.5 (Beta)

- Mejoras de UI: Navegación por pestañas con botones de avance automático.
- Corrección Interactiva: Nuevo chatbot que permite dialogar con la IA para resolver errores de validación. Implementación de una "Caja de Sugerencias" para revisar y aplicar cambios al XML de forma segura.
- Logo Embedded: El logo de la revista ahora se incrusta como Base64 en el HTML generado, eliminando la dependencia de carpetas locales.
- Optimización: Eliminación de animaciones intrusivas y mejora en la legibilidad del chat en modo oscuro.

v0.1 v0.4 (Alpha)

- Inicio del proyecto.
- Configuración de Gemini CLI.
- Extracción básica de DOCX.
- Primera implementación de transformación a XML JATS.

Guía de Contribución

Contribuir al Proyecto

¡Gracias por tu interés en mejorar el Transformador XML JATS! Este documento describe las normas y mejores prácticas para contribuir al código.

Configuración del Entorno de Desarrollo

1. Clonar el repositorio:

```
git clone <URL_DEL_REPOSITORIO>
cd <NOMBRE_DEL_DIR>
```

2. Crear un entorno virtual (Python 3.9+ recomendado):

```
python -m venv .venv
source .venv/bin/activate # En Windows: .venv\Scripts\activate
```

3. Instalar dependencias:

```
pip install -r requirements.txt
```

4. Configurar la API Key:

Ingresa tu clave GEMINI_API_KEY desde la pestaña " Configuración" en la interfaz web, o exporta la variable de entorno antes de ejecutar:

```
export GEMINI_API_KEY="tu_clave_aqui"
streamlit run streamlit_app.py
```

> Una sola clave: el sistema no usa ni necesita una segunda key. Si la cuenta tiene facturación habilitada en Google Cloud, la misma clave funciona en el tier de pago al agotar la cuota gratuita.

Arquitectura del Proyecto

El proyecto sigue una arquitectura desacoplada donde el backend es independiente de la interfaz gráfica:

```
streamlit_app.py      # Punto de entrada web
```

views/

transformador.py # UI principal (Streamlit)

configuracion.py # Configuración de API key y tokens

manual_usuario.py # Documentación y manual

modules/

transformer.py # Lógica de conversión DOCX XML

prompts.py # Plantillas de prompts para la IA

metadata_processor.py # Extracción de metadatos

correction.py # Corrección asistida por IA

xml_html.py # Conversión XML HTML

config_store.py # Persistencia (SQLite) una API key

Principios de Diseño

- Backend sin dependencia de UI: transformer.py, metadata_processor.py, correction.py y prompts.py NO deben depender del estado de streamlit. La configuración se pasa como parámetros y se guarda usando config_store.py.
- Prompts Centralizados: Todas las instrucciones para la IA están en prompts.py. Nunca hardcodees prompts en otros módulos.
- Modelo Consistente: El modelo default es gemini-2.5-flash en toda la aplicación. Si añades un nuevo punto de llamada a la IA, usa este modelo.
- Una sola API Key: No añadas parámetros api_key_pro ni segundas claves. Cuando la cuota gratuita se agota, transformer.py y correction.py retornan {'quota_exceeded': True} y la UI muestra el banner correspondiente. Google maneja el tier de pago automáticamente con la misma clave.
- Señalización de Cuota: Si una función hace llamadas a Gemini y puede recibir error 429, debe retornar un diccionario que incluya 'quota_exceeded': True (no lanzar excepción), para que la UI detecte el estado y actualice el banner lateral.

Estilo de Código y Normas

Para mantener la calidad y legibilidad del código, exigimos el cumplimiento de las siguientes normas en todos los Pull Requests:

1. Convenciones Generales

- Seguimos PEP 8.
- Longitud de línea máxima sugerida: 100 caracteres.
- Usa snake_case para funciones y variables, y PascalCase para clases.

2. Type Hints (Tipado Estático)

Todas las funciones y métodos deben tener anotaciones de tipo (Type Hints) completas para argumentos y valores de retorno.

Correcto:

```
def procesar_texto(entrada: str, opciones: Dict[str, bool]) -> List[str]:  
    ...
```

Incorrecto:

```
def procesar_texto(entrada, opciones):  
    ...
```

3. Docstrings (Estilo Google)

Todas las funciones, clases y módulos públicos deben tener docstrings siguiendo el Google Python Style Guide.

Ejemplo:

```
def funcion_ejemplo(arg1: int, arg2: str) -> bool:  
    """Realiza una operación de ejemplo.
```

Explica brevemente qué hace la función. Puede tener múltiples líneas si es complejo.

Args:

arg1 (int): El primer argumento, que representa X.

arg2 (str): El segundo argumento, que representa Y.

Returns:

bool: True si la operación fue exitosa, False en caso contrario.

Raises:

ValueError: Si arg1 es negativo.

```
    """  
  
    ...
```

4. Manejo de Respuestas de IA

Al procesar respuestas de la IA (Gemini), sigue estas reglas:

- Parsing de JSON: Siempre busca el primer { y último } para extraer el JSON, ya que el modelo puede anteponer texto.
- Parsing de XML: Verifica si la respuesta comienza con < (XML directo) o contiene bloques `<xml>` (markdown). invocar `_gemini_cli` ya limpia backticks via `parse_model_response`.
- Normalización: Usa `_normalize_metadata()` para corregir variaciones en claves del JSON.
- Cuota agotada: Detecta el error 429 con tenacity; si se agotan los reintentos, retorna `{'error': ..., 'quota_exceeded': True}`.

5. Prompts para la IA

Al modificar o añadir prompts en `prompts.py`:

- Incluye instrucciones explícitas para DOI, fechas y secciones con títulos.
- Para tablas, incluye la estructura XML completa como ejemplo.
- Usa negritas/mayúsculas para reglas CRÍTICAS que el modelo suele ignorar.
- Siempre incluye recordatorios sobre cierre correcto de etiquetas XML.

Flujo de Trabajo

1. Reportar Problemas: Abre un Issue antes de comenzar trabajos grandes.
2. Ramas (Branches): Crea ramas descriptivas, ej. `feature/validacion-xml` o `fix/error-parseo`.
3. Pull Requests:
 - Describe claramente tus cambios.
 - Asegúrate de que el código pase las validaciones locales (`python3 -c "import ast; ast.parse(open('archivo.py').read())"` como mínimo).
 - Adjunta capturas de pantalla si cambiaste la interfaz gráfica.

Errores Conocidos y Mitigaciones

| Problema | Mitigación |

| --- | --- |

| La IA antepone texto antes del JSON de metadatos | Extracción por {...} en `metadata_processor.py` |

| La IA no cierra etiquetas XML correctamente | Instrucciones reforzadas en `prompts.py` + sanitización en `transformer.py` |

max_output_tokens insuficiente para artículos largos	Configurado a 65 536 tokens
Backticks eliminados por parse_model_response	Detección dual de XML (directo o con backticks) en transformador.py
Cuota gratuita agotada (HTTP 429 persistente)	transformer.py retorna quota_exceeded: True; la UI muestra el banner automáticamente

Contacto

Para dudas técnicas, contacta a: cristian.carreno@uv.cl
Universidad de Valparaíso.

Manual de Usuario

El Transformador XML JATS es una herramienta especializada diseñada para optimizar y automatizar el flujo de trabajo editorial de la Universidad de Valparaíso.

Esta solución permite la conversión de manuscritos originales en formato Microsoft Word (.docx) al estándar JATS XML (Journal Archiving and Interchange Tag Suite), asegurando el cumplimiento con los requisitos de indexación y preservación digital de alto nivel.

Características principales:

- Extracción automática de metadatos (título, autores, DOI, fechas).
- Conversión de tablas a formato JATS con estructura completa.
- Generación de HTML con DOIs clickeables y referencias bien formateadas.
- Validación contra el estándar JATS 1.4 (ANSI/NISO Z39.96-2024).

Flujo de Trabajo

Configuración Inicial

Antes de comenzar, diríjase a la pestaña " Configuración" en el menú izquierdo para preparar su entorno:

1. Clave de API Gemini:

- Primera vez: Pegue su clave y presione " Guardar". Se almacenará de forma segura y persistente.
- Sesiones posteriores: La clave se carga automáticamente (ej. AlzaxY4Z).

2. Metadatos de Publicación y Estándar:

- Debajo del panel de cuotas encontrará los Metadatos Persistentes de la Revista.
- Rellene aquí el Título de Revista, la Editorial, el ISSN, y seleccione qué Versión DTD JATS utilizará (1.3 o 1.4).
- Estos datos se autocompletarán de forma transparente en cada validación que realice en la herramienta, ahorrando trabajo repetitivo.

3. Panel de Consumo:

- Encontrará una barra de progreso que ilustra las Requests usadas hoy vs. el límite diario, junto al historial y advertencias preventivas si se acerca al tope de su cuota gratuita.

1. Carga de Archivos

- Navegue a la sección "Transformador" en el menú lateral.
- Encontrará un área designada para la carga de archivos.
- Puede subir documentos en formato WORD (.docx) o PDF (.pdf).

Una vez cargado el archivo, el sistema desplegará el panel de Revisión de Metadatos:

- Verifique los datos: Título, Revista, Fecha, DOI, autores, etc.
- Complete lo faltante: Si faltan datos clave (el DOI y la fecha son obligatorios), un Chatbot Asistente aparecerá para pedirselos. Puede escribirlos en el chat o en los campos editables.
- Revise también la vista previa del texto extraído, incluyendo las tablas detectadas.

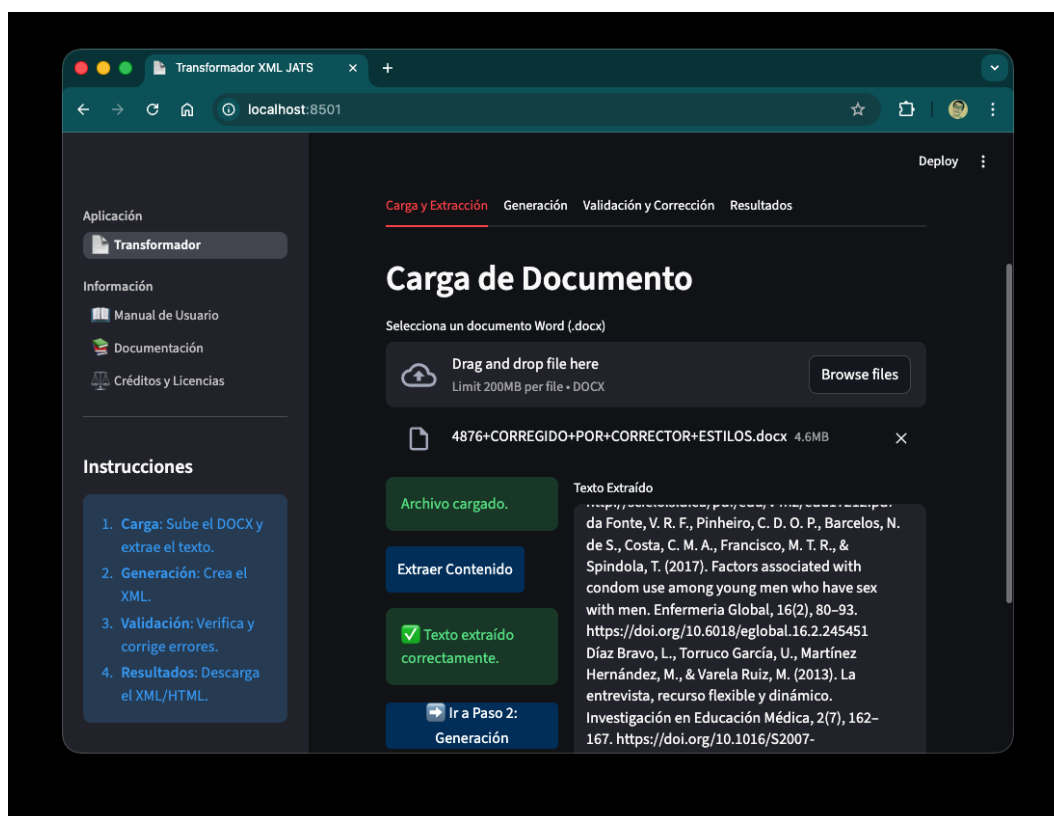


Figura 2: Área de carga de documentos y extracción de metadatos.

2. Generación de XML JATS

Una vez confirmados los metadatos y el contenido, en la pestaña "Generación":

- Presione el botón "Generar XML JATS".
- El sistema procesará el contenido aplicando las reglas del último estándar JATS 1.4 (ANSI/NISO

Z39.96-2024).

- Se utilizarán los metadatos validados para construir un encabezado (<front>) preciso, incluyendo DOI y fechas.
- Las tablas del documento se convertirán automáticamente a <table-wrap> con encabezados y cuerpo.
- Cada sección del artículo (Introducción, Métodos, Resultados, etc.) se mapeará a su etiqueta <sec> correspondiente.

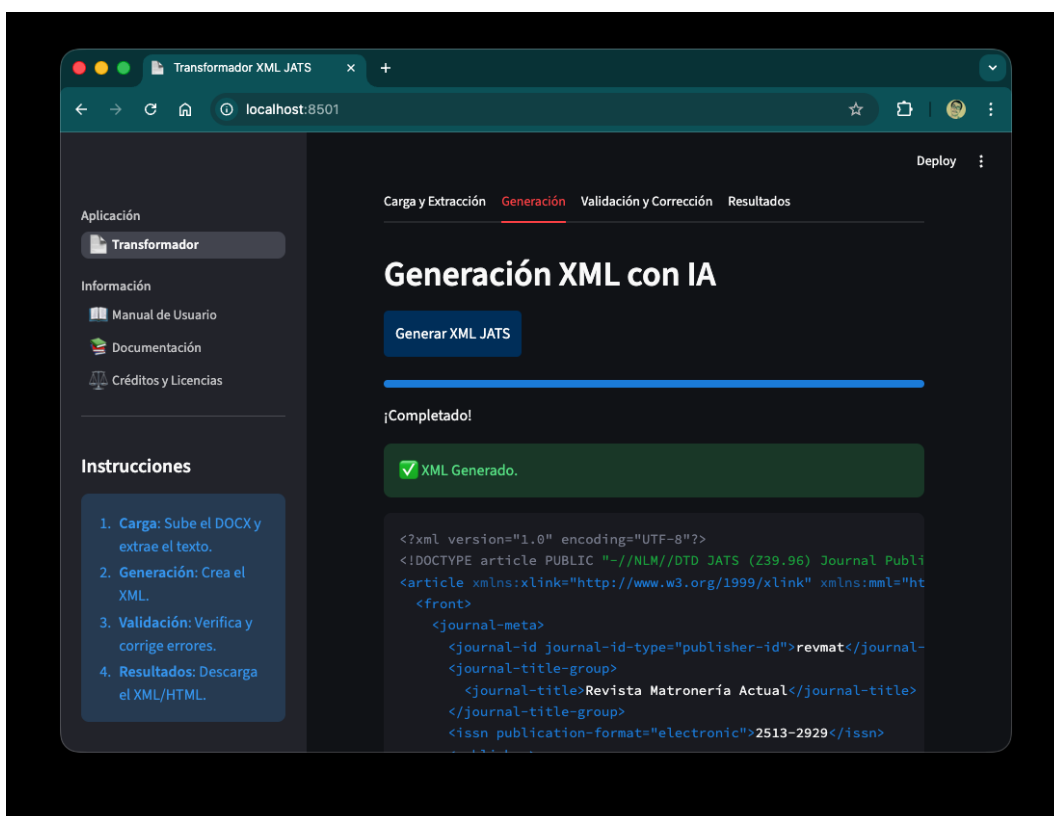


Figura 3: Generación exitosa de XML JATS.

3. Validación y Corrección

- Presione el botón "Ejecutar Validación DTD".
- Se ejecutará una validación automática estricta contra el DTD oficial JATS 1.4.

Importante

- Si el sistema encuentra errores, se mostrará un informe con los errores encontrados y se habilitarán un botón de corrección que permitirá corregir los errores encontrados asistido por la IA de Gemini.
- Si el sistema no encuentra errores, se mostrará un mensaje de éxito y se habilitarán el botón de Resultados.

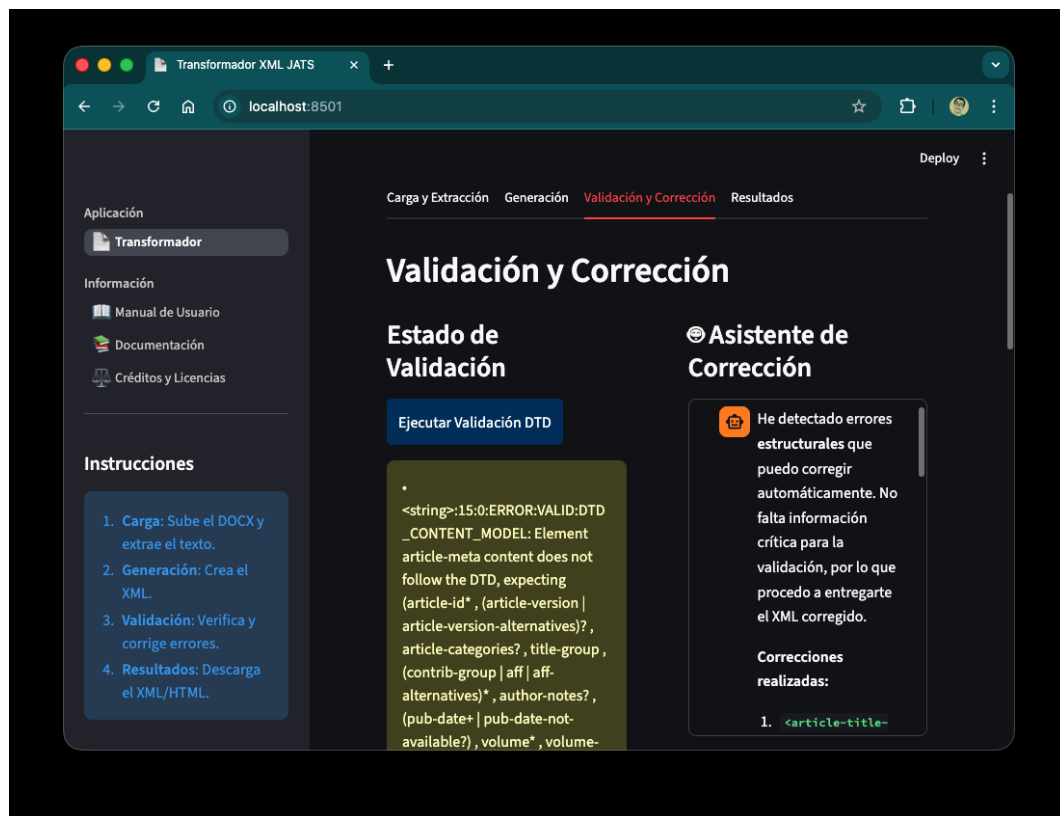


Figura 4: Validación de DTD ejecutada sin errores.

4. Resultados

Si la validación es exitosa, se habilitarán los botones de descarga:

- Descargar XML: Archivo listo para publicación/preservación.
- Descargar HTML: Vista previa para lectura web con DOIs clickeables.
- Vista previa HTML: Haga clic en el botón "Ver vista previa en nueva pestaña" para abrir el HTML en una pestaña independiente del navegador. Los enlaces internos (citas, tabla de contenidos) y los DOIs en las referencias funcionan correctamente en esta vista.

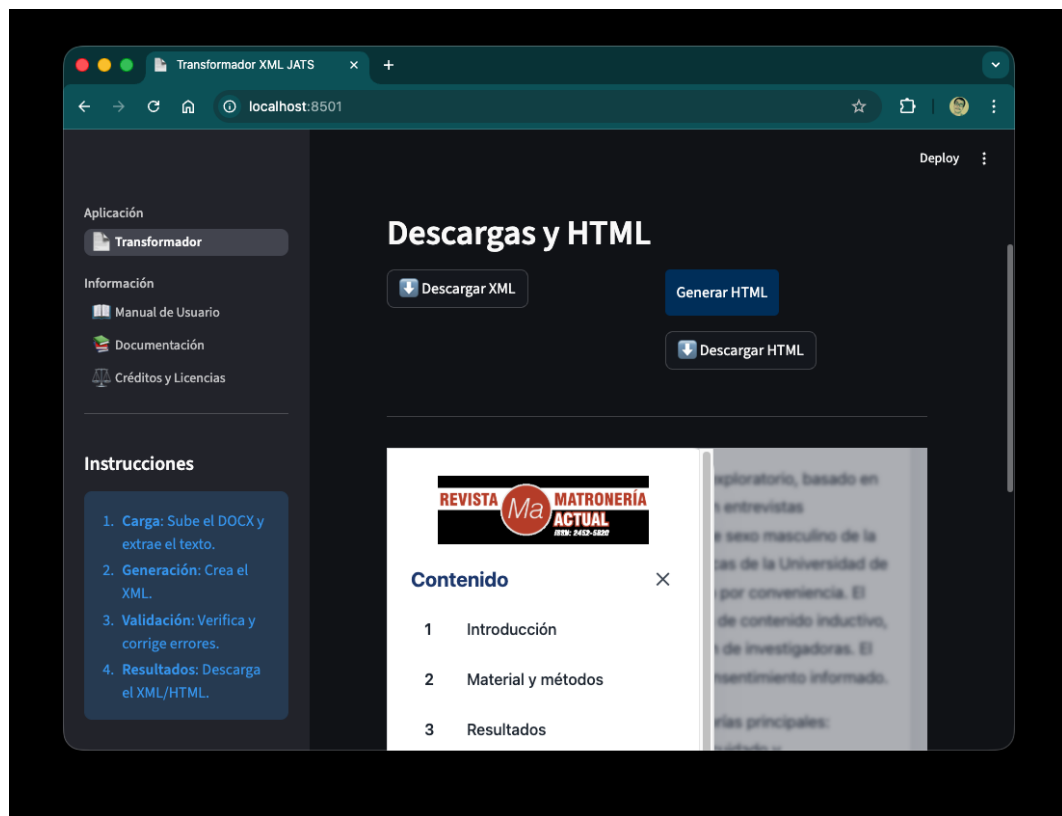


Figura 5: Resultados y enlaces de descarga/vista previa.

Preguntas Frecuentes

¿Qué formatos soporta?

Soporta archivos Microsoft Word (.docx) y documentos Portables (.pdf).

¿Qué hago si falla la validación?

Revise el mensaje de error. Generalmente se debe a caracteres especiales no soportados o estructuras de documento inusuales (ej. tablas anidadas complejas). Puede usar el botón "Intentar Solucionar con IA" para que Gemini corrija automáticamente los errores, o editar el XML manualmente.

¿Las tablas del documento se incluyen en el XML?

Sí. Las tablas se extraen automáticamente y se convierten a formato JATS (<table-wrap>) con encabezados (<thead>) y cuerpo (<tbody>) correctamente estructurados, preservando todas las filas y columnas del documento original.

¿Se incluye el DOI y la fecha de publicación?

Sí. El DOI y la fecha de publicación son campos obligatorios que se extraen automáticamente de los metadatos. Si el sistema no los detecta, le pedirá que los ingrese antes de generar el XML.

¿Los enlaces del HTML funcionan correctamente?

Sí. Los enlaces de citas (ej. [1], [2]) y la tabla de contenidos navegan correctamente dentro de la página. Los DOIs en las referencias bibliográficas son clickeables y enlazan directamente a doi.org. Use siempre el botón "Ver vista previa en nueva pestaña" para la mejor experiencia.

¿Los caracteres acentuados se ven bien?

Sí. El sistema decodifica correctamente todos los caracteres UTF-8 (á, é, í, ó, ú, ñ, etc.) tanto en la vista previa como en el archivo descargado.

¿Es seguro subir mis archivos?

Sí, los archivos se procesan temporalmente en la memoria para su transformación y no se guardan en el servidor de forma permanente.

¿Mi API Key está segura?

Sí. La clave se almacena localmente en una base de datos SQLite (carpeta data/) con ofuscación Base64. Una vez guardada, desaparece de la interfaz y solo se muestra enmascarada. La carpeta data/ está excluida del control de versiones.

¿Qué pasa si alcanzo el límite de tokens gratuitos?

El panel de consumo en la pestaña " Configuración" muestra su consumo actual vs. los límites del tier gratuito de cada modelo. Al alcanzar el 80%% se muestra una advertencia; al 100%% las solicitudes podrían fallar. Puede esperar al día siguiente (los límites diarios se renuevan) o cambiar a un modelo con mayor

cuota.

Créditos y Licencias

Esta herramienta ha sido desarrollada para optimizar el flujo de trabajo editorial de las Revistas UV, automatizando la conversión de manuscritos a XML JATS validado.

Desarrollador Principal:

Cristian Carreño León (cristian.carreno@uv.cl)

Escuela de Obstetricia y Puericultura

Facultad de Medicina

Universidad de Valparaíso, Chile

Tecnologías Utilizadas:

- Python 3.9+: Lenguaje base.
- Streamlit: Framework de interfaz de usuario web.
- Google Gemini 2.5 Flash: Modelo de lenguaje (LLM) para etiquetado inteligente.
- lxml: Procesamiento, validación y parsing de XML/HTML.
- python-docx: Extracción de contenido desde archivos Word.
- FPDF2: Generación de manuales en PDF.
- JATS 1.4 (ANSI/NISO Z39.96-2024): Estándar de etiquetado XML.

Privacidad y Tratamiento de Datos:

1. Procesamiento Volátil: Los archivos no se almacenan permanentemente en el servidor.
2. API de Inteligencia Artificial: Se utiliza Google Gemini bajo sus términos de servicio. El texto enviado a la API se procesa bajo las políticas de privacidad de Google.

Licencia:

Software de uso exclusivo para la Universidad de Valparaíso. Todos los derechos reservados.

El código fuente y la documentación son propiedad intelectual de la Universidad de Valparaíso y su autor.

Versión: 0.68